

《算法分析与设计》第 3 次作业 *

姓名：谢鹏宇 学号：71122135 成绩：_____

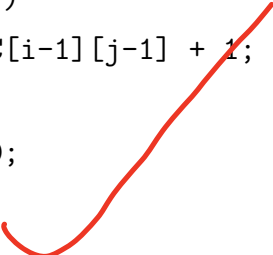
算法分析题

题目 1：一个字符串中的连续的一段字符序列称为该字符串的一个子串。例如 *school* 中 *cho* 就是一个子串。子串和子序列的区别是，子序列在原序列中的位置不一定连续，而子串必须连续。两个字符串都含有的子串称为它们的公共子串。例如，*university* 和 *anniversary* 就含有许多公共子串，例如 *ni, niv, y, er* 等，但最长的一个是 *nivers*。假设有字符串 $X = x_1x_2...x_n$ 和 $Y = y_1y_2...y_m$ ，请设计一个基于动态规划的 $O(mn)$ 算法来找出 X 和 Y 的最长公共子串。（要写出最长公共子串在字符串 X 的起止位置）

答：

```
void maxString(string x, int n, string y, int m){
    int C[n][m] = {0};

    for(int i=0; i<n; ++i){
        for(int j=0; j<m; ++j){
            if(i==0 || j==0){
                if(x[i] == y[j])
                    C[i][j] = 1;
                else
                    C[i][j] = 0;
            }
            else{
                if(x[i] == y[j])
                    C[i][j] = C[i-1][j-1] + 1;
                else
                    C[i][j] = 0;
            }
        }
    }
}
```



*要求：1、分析题请用书面化语言给出详细分析过程。

```

}

int last=1, length=0;

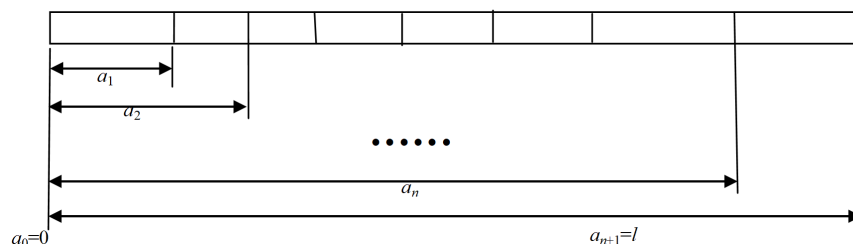
for(int i=0; i<n; ++i){
    for(int j=0; j<m; ++j){
        if(C[i][j] > length){
            length = C[i][j];
            last = i;
        }
    }
}
}
}

```

最终起始位置为 $X_{last-length}$, 终止位置为 X_{last} ;



题目 2: 假设我们要将一根大型长钢管锯成若干段。我们在要锯的地方打上标记，一共是 n 个标记。这些标记距离钢管左端的距离，从左到右，分别是 $a_1, a_2, \dots, a_n$ 厘米。这个钢管的总长是 l 厘米， $l > a_n$ (参见下图)。当我们将钢管锯为两截时，需要的代价与当时被锯钢管的长度成正比，比例是每一厘米 p 元。



- (a) 请用动态规划的方法设计一个算法来找出最优的顺序来完成这 n 处的切割使总的代价最小。分析你的算法的复杂度。(提示：用 $a_0 = 0$ 和 $a_{n+1} = l$ 表示钢管左端和右端位置。用 $[a_i, a_j]$ 表示从标记 a_i 到标记 a_j 这段钢管。用 $C(i, j)$ 表示完成对 $[a_i, a_j]$ 这段钢管的切割任务所需的最小代价，也就是完成所有 $a_k (i < k < j)$ 处的切割所需代价。找出 $C(i, j)$ 的归纳公式。)
- (b) 以下面数据为例，用你在 (a) 中的算法找出最优的切割顺序和总代价。请显示你的计算过程。

$$a_1 = 2, a_2 = 5, a_3 = 9, a_4 = 14, l = 15, p = 1.$$

答: a)

$$C(i, j) = \begin{cases} 0 & i = j \text{ 或 } j = i + 1; \\ p * (a_j - a_i) & j = i + 2; \\ \min_{i < k < j} \{C(i, k) + C(k, j)\} + p * (a_j - a_i) & j > i + 2; \end{cases}$$

```
void minTired(int k){
    int C[n+1][n+1] = {0};

    for(int i=1; i<n+1; ++i){
        C[i][i+2] = a_i;
    }

    for(int i=0; i<n+1; ++i){
        for(int j=i+1; j<=n+1; ++j){
            for(int k=i; k<j; ++k){
                C[i][j] = min_{i<k<j} {C[i][k]+C[k][j]}+p*(a_j-a_i);
            }
        }
    }
}
```

时间复杂度为 $O(n^3)$

b)

- 先切割 a_3 , 费用为 15 元
- 再切割 a_4 , 费用为 6 元
- 再切割 a_2 , 费用为 9 元
- 再切割 a_1 , 费用为 5 元

总费用共计 35 元

为什么是这个顺序?

-5

题目 3: 搬东西是很累的, 帕鲁深有体会。看着箱子里的 n 件物品, 帕鲁开始发呆, 因为 n 是一个小于 2000 的整数, 实在是太多了, 于是帕鲁决定随便搬 $2 * k$ 件过去就行了。但还是会很累, 因为 $2 * k$ 也不小是一个不大于 n 的整数。幸运的是帕鲁根据多年的搬东西的经验发现每搬一次的疲劳度是和左右手的物品的重量差的平方成正比 (帕鲁每次搬两件东西, 左手一件右手一件)。例如帕鲁左手拿重量为 3 的物品, 右手拿重量

为 6 的物品, 则他搬完这次的疲劳度为 $(6 - 3)^2 = 9$ 。现在可怜的帕鲁希望知道搬完这 $2 * k$ 件物品后的最佳状态是怎样的 (也就是最低的疲劳度), 请用动态规划的方法告诉他吧。(提示: 令 $dp(i, j)$ 表示从前 i 个物品选出 j 对物品的最低疲劳度, 若 $j * 2 = i$, 则 $dp(i, j) = dp(i - 2, j - 1) + (w[i] - w[i - 1])^2$, 那么 $j * 2 \neq i$ 时呢?) (要求给出分析和伪代码)

答: 分析:

$$dp(i, j) = \begin{cases} 0 & i = 0, j = 0; \\ w[1]^2 & i = 1, j = 0; \\ dp(i - 2, j - 1) + (w[i] - w[i - 1])^2 & i = 2 * j; \\ dp(i - 2, j) + w[i]^2 & i \neq 2 * j; \end{cases}$$

代码:

```
void minTired(int k){
    int dp[n][m] = {0};

    for(int i=0; i<k; ++i){
        if(i==0 && j==0){
            dp[i][j] = 0;
        }
        else if(i==1 && j==0){
            dp[i][j] = w[1]^2;
        }
        else if(i = 2*j){
            dp[i][j] = dp(i-2,j-1)+(w[i]-w[i-1])^2;
        }
        else{
            dp[i][j] = dp(i-2,j)+w[i]^2;
        }
    }
}
```

-15

最终结果为 $dp[2k][k]$